

Patterns Principali

Decomposition Pattern

cosa sono i pattern dei microservizi. Prima di entrare nel pattern, il docente ricorda che i microservizi, parlando in generale, sono descritti da una serie di **pattern** che affrontano l'architettura e tutte le problematiche legate a questo tipo di applicazioni. E ricorda tre caratteristiche di base che vanno tenute a mente:

- ogni microservizio ha **un suo server**;
- ogni microservizio ha **un proprio database**;
- i microservizi **non devono essere né troppo grandi né troppo piccoli**. La regola pratica che dà: la dimensione giusta è quella per cui ogni microservizio può essere sviluppato da un **team di 10–15 persone**, in modo da poter applicare bene le metodologie **agile**.

Definizione del pattern. Il *Decomposition Pattern* è il pattern che ci dice **come decomporre un'applicazione monolitica in più microservizi**. Cioè: partiamo da un'unica applicazione monolite e la scomponiamo in tanti microservizi. Il punto centrale non è tanto "spezzare", ma **evitare una decomposizione sbagliata**: una decomposizione errata porta con sé tante problematiche e — questa è la frase chiave della lezione — può addirittura risultare **peggiore del monolite di partenza**.

I modi per decomporre sono tanti, ma quelli più utilizzati sono **due**: per *business capabilities* e per *dominio*.

Decomposizione by Business Capability

Business capability si traduce come **capacità aziendale**: è **ciò che un business fa per generare valore**. Ogni azienda ha più business capability — per esempio le vendite, il servizio clienti, il marketing — e ovviamente variano in base al settore in cui l'azienda opera. Spesso queste capacità **corrispondono ai reparti/settori** in cui l'azienda è suddivisa.

In ambito sviluppo software, le business capability si possono vedere come i **concetti e le funzionalità di alto livello**, cioè quelle funzionalità che generano il maggior valore per l'applicazione.

L'esempio della lezione: la piattaforma sanitaria. Consideriamo una piattaforma sanitaria che fornisce vari servizi. Le sue business capability sono:

- gestione dei **pazienti**;
- **schedulazione degli appuntamenti**;
- gestione delle **cartelle cliniche**;
- **billing**, cioè i pagamenti e i vari processi di pagamento;
- **telemedicina**.

Ognuna di queste funzionalità può essere gestita tramite un microservizio: quindi **a ogni funzionalità associamo un microservizio**.

Il criterio in una frase. La decomposizione by business capability suddivide il progetto **in base a quello che si vuole fare**. Ogni microservizio ha una sua "zona" che rappresenta quello che deve

produrre e quello che deve gestire: ci si concentra molto sulle **capacità**, su quello che il servizio deve eseguire.

Decomposizione by Subdomain (per dominio)

Il docente introduce il secondo approccio notando che in certi contesti è **complicato capire bene cosa deve fare ogni cosa** — e lì la prima strada diventa difficile da applicare.

Definizione. Il **dominio** di un'applicazione è **l'ambito specifico in cui l'applicazione opera**. Per esempio, in un sistema per la gestione di una libreria online il dominio è la gestione e la vendita di libri online.

L'approccio prevede di suddividere il dominio dell'applicazione in **sottoinsiemi più piccoli, chiamati sottodomini**, ognuno dei quali rappresenta **un'area funzionale ben definita**. A ogni sottodominio corrisponde un microservizio.

L'esempio della lezione: l'applicazione assicurativa. Immaginiamo un'applicazione monolitica assicurativa che consente ai clienti di acquistare un'assicurazione online. Per implementarla a microservizi la suddividiamo in sottodomini:

- **Sales** (vendite) — gestisce l'acquisto della polizza. Questo dominio a sua volta si può suddividere in **due sottodomini**: quello che riguarda le **vendite** vere e proprie e quello che gestisce i **reclami** (claims). Quindi: un servizio per l'acquisto e un servizio per i reclami.
- **Customer** — serve a gestire i clienti, le loro informazioni e le interazioni con loro.
- **Compliance** — il servizio di conformità, che garantisce che la nostra assicurazione sia conforme alle varie normative.
- **Marketing** — anche questo si suddivide a sua volta in sottodomini:
 - **Campagne**, per creare le campagne pubblicitarie legate al marketing;
 - **Analisi**, che permette di tracciare e analizzare i dati del marketing;
 - **Report**, che permette di generare report sulle prestazioni del marketing.

Nota il punto importante che emerge dall'esempio: **i sottodomini sono gerarchici**. Un sottodominio (Sales, Marketing) può essere ulteriormente scomposto in sottodomini più fini, e la scomposizione si ferma quando l'area funzionale è "ben definita".

Le due decomposizioni a confronto

Ci sono anche altri modi di decomporre un monolite, ma questi due sono i più usati. Il confronto che fa il docente:

Quale scegliere? Dipende dal caso specifico. L'esempio pratico che dà: se **non conosciamo bene la logica di business**, decomporre secondo le competenze chiave (business capability) diventa problematico, e quindi preferiamo decomporre in base alle varie aree/tematiche di conoscenza — oppure viceversa.

Riepilogo della lezione. Il Decomposition Pattern ci dice i vari modi in cui possiamo decomporre un'applicazione monolite in tanti microservizi. I due principali: **business capabilities** (decomposizione in base a quello che l'applicazione fa e produce) e **dominio** (decomposizione in base ai concetti e alle aree di conoscenza dell'applicazione). Quale applicare dipende dal caso specifico.

- **Nelle piccole applicazioni** le due decomposizioni finiscono per essere **quasi identiche**: portano più o meno allo stesso risultato.
 - **La differenza si fa sentire nelle grandi applicazioni**:
 - con le **business capabilities** ci si concentra soprattutto su **che cosa si deve fare**, su quello che si deve **produrre**;
 - con il **dominio** ci si concentra sui **concetti** e sulle **funzionalità specifiche del dominio del problema** che stiamo trattando, cioè sulle **aree di conoscenza**.
-

Saga Pattern

Il problema: la consistenza dei dati nelle transazioni distribuite

Com'era nel monolite. Con un'applicazione monolitica il problema della consistenza dei dati sostanzialmente non esisteva: dovevi creare un ordine, facevi `start transaction`, eseguivi le tue operazioni sulle varie tabelle del database, poi chiudevi la transazione — ed eri sicuro che a fine transazione i dati erano **corretti e consistenti**. E se succedeva qualcosa, andava tutto in **rollback** e "amici come prima": il database veniva riportato allo stato iniziale. Quindi la transazione non comportava particolari problematiche.

Perché nei microservizi non funziona più. Nel mondo dei microservizi **ogni microservizio in genere si occupa di una sua tabella** (ha il proprio database). Magari abbiamo venti microservizi e dobbiamo portare a termine **una transazione che coinvolge tanti attori**, cioè tanti microservizi diversi. Non c'è più un'unica transazione su un unico database — e questo porta problemi di consistenza.

Il primo tentativo: il pattern a due fasi (2PC)

Come funziona. C'è una figura centrale, il **coordinatore**. Il flusso descritto:

1. l'utente dice al coordinatore che vuole creare l'ordine;
2. il coordinatore **crea una transazione** e chiama il *Customer Service*;
3. poi chiama l'altro microservizio, l'*Order Service*;
4. **attende la risposta di entrambi** i microservizi;
5. modifica quello che deve modificare, invia il risultato al Customer Service e poi anche all'*Order Service*;
6. **termina la transazione**.

Quindi c'è questa figura centrale che coordina tutti i vari microservizi. Il problema delle transazioni distribuite *sembrerebbe* risolto. In realtà non è così.

I problemi del 2PC.

- **Collo di bottiglia.** Per terminare la transazione bisogna aspettare che **tutti** i microservizi abbiano terminato il loro compito. Se devo chiamare sei microservizi, devo attendere che tutti e sei abbiano finito: il coordinatore chiama un microservizio, rimane in attesa, ne chiama un altro, rimane in attesa... deve avere tutte le risposte prima di poter fare il **commit**.
- **Indisponibilità di un servizio.** Se un microservizio non è più disponibile "succede il panico": bisogna attendere, e si ha un **tempo di rollback elevato**.

- **Non è scalabile.** All'aumentare della complessità aumenta anche la complessità del coordinatore stesso.

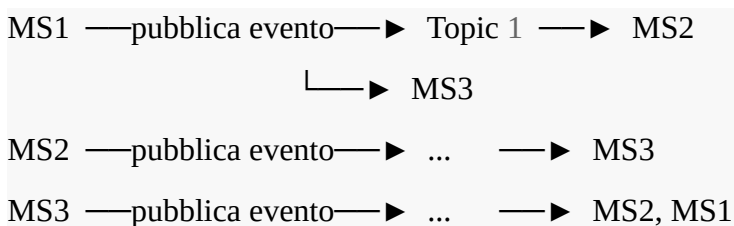
Per questi motivi il 2PC è stato sostituito dal **pattern Saga**, che ha due tipi di implementazione: **basata sulla coreografia** e **basata sull'orchestrazione**.

(Sul nome: il docente scherza dicendo che "Saga" sembra il nome di un videogioco — e poi spiega che in effetti il pattern è come una saga, come una serie TV: una **serie di eventi concatenati**.)

Saga con coreografia (choreography)

La logica è **affidata agli eventi**: ogni microservizio **aspetta un evento** per poter partire ed eventualmente **invia altri eventi** agli altri microservizi coinvolti. Non c'è nessuno che coordina.

L'esempio della lezione (flusso di eventi).



Il microservizio 1 pubblica un evento sul **topic 1**, che a sua volta viene letto dal microservizio 2 e dal microservizio 3. Poi il microservizio 2 invia un altro evento, letto dal microservizio 3; e il microservizio 3 a sua volta ne invia un altro, letto dal 2 e dall'1.

Il concetto chiave: **ognuno sa precisamente, nel suo piccolo, qual è l'evento scatenante, cosa deve fare quando arriva quel determinato evento e quali eventuali eventi inviare**. Non c'è un coordinatore centrale: la logica è **decentralizzata**.

Vantaggi.

- **Nessun single point of failure di coordinamento.** Nel 2PC, se crollava il coordinatore non funzionava più niente. Qui invece tutto continua a funzionare, perché un coordinatore centrale non c'è.
- **Molto adatto quando i microservizi sono pochi.**
- **Molto adatto ai team agile**, perché ogni team può sviluppare il proprio microservizio in autonomia.

Svantaggi.

- Quando gli attori diventano tanti, **la complessità aumenta tantissimo**.
- Si corre il rischio di creare **cicli**: il microservizio 3 aspetta l'evento del 2, che a sua volta aspetta l'evento dell'1... e si rischia di bloccare tutto.
- Con tanti attori **non si capisce più bene chi fa cosa**: diventa complicato da gestire.

Saga con orchestrazione (orchestration)

Esiste proprio per risolvere i limiti della coreografia. Qui c'è un elemento che coordina tutti: l'**orchestratore**, una specie di coordinatore che coordina i vari microservizi — ma attenzione, **sempre tramite un message broker, sempre tramite eventi**. L'orchestratore **manda il comando tramite un evento e rimane in attesa dell'evento di risposta** del microservizio.

Vantaggi.

- Abbiamo un **punto centrale che coordina tutti**, quindi il **rollback è più semplice**.
- Si **evitano i cicli**.
- Resta l'**approccio agile**: ogni team può lavorare al proprio microservizio, perché ne conosce l'evento scatenante e l'evento che deve produrre.
- È più semplice **capire chi deve fare cosa**.

Svantaggi.

- **Singolo punto di rottura**: se l'orchestratore muore, l'applicazione crolla — esattamente come nell'altro caso (il 2PC).
- **Il numero di messaggi è duplicato**: l'orchestratore invia un messaggio di comando e deve comunque ricevere un evento di risposta, quindi il numero di interazioni è praticamente **raddoppiato**.

Quale delle due usare

Dipende, e il criterio è il **numero di microservizi coinvolti**:

- **pochi microservizi** → conviene la **coreografia**;
- **tanti microservizi**, con transazioni distribuite che coinvolgono tantissimi attori → conviene l'**orchestrazione**.

In **entrambi i casi** però avremo sempre una **logica basata sugli eventi** e un **message broker**. Il docente insiste su questo punto: i message broker in un'architettura a microservizi sono una componente molto importante.

Riepilogo della lezione. Il problema è gestire le transazioni distribuite tra più microservizi. Un primo modo era il **pattern a due fasi**, che però ha un punto di rottura e un collo di bottiglia, perché ogni microservizio deve attendere gli altri. Per questo si usa spesso il **pattern Saga**, in due versioni: la **coreografia**, dove c'è sempre un message broker, ogni microservizio conosce l'evento scatenante e l'evento da produrre e nessuno coordina (logica decentralizzata) — ottima con pochi servizi, ma al crescere degli attori non si capisce più chi fa cosa e si rischiano cicli e blocchi reciproci; e la versione con **orchestratore**, dove esiste sempre il message broker e il ragionamento a eventi, ma c'è un unico orchestratore: più semplice capire chi fa cosa, però restano il singolo punto di fallimento e il raddoppio dei messaggi scambiati.

Il docente chiude precisando che questa è una **breve introduzione** al pattern Saga e che per approfondire si rimanda alla documentazione ufficiale.